Data-Driven Fluid Dynamics - Assignment 1

Casey Allen

24th February 2026

Contents

1 Task A: Binary Classification 3
- 1.1 Comparative Experiment . . . 3
  - 1.1.1 Updated plot snippet . . . 3
  - 1.1.2 Training and Validation loss plot . . . 3
  - 1.1.3 Test loss snippet . . . 4
  - 1.1.4 Test loss output . . . 4
- 1.2 Discussion . . . 4

2 Task B: Multiclass Classification 6
- 2.1 Increasing Class Density . . . 6
  - 2.1.1 Data preparation and training changes snippet . . . 6
  - 2.1.2 Evaluation snippet . . . 6
  - 2.1.3 Output . . . 6
  - 2.1.4 Discussion . . . 7
- 2.2 Increasing Data Noise (Standard Deviation) . . . 7
  - 2.2.1 Updated data code snippet . . . 7
  - 2.2.2 Output . . . 8
  - 2.2.3 Loss Plot . . . 8
  - 2.2.4 Discussion . . . 8
- 2.3 High-Dimensional Input Space . . . 8
  - 2.3.1 Altered code snippet . . . 8
  - 2.3.2 Output . . . 9
  - 2.3.3 Loss Plot . . . 9
  - 2.3.4 Discussion . . . 9

3 Task C: Model Architecture (Regression) 10
- 3.1 Activation Function Impact . . . 10
  - 3.1.1 Code snippet . . . 10
  - 3.1.2 Output . . . 11
  - 3.1.3 Loss Plot . . . 11
  - 3.1.4 Discussion . . . 11
- 3.2 Increasing Model Depth . . . 12
  - 3.2.1 Code snippets . . . 12
  - 3.2.2 Output . . . 12
  - 3.2.3 Loss Plot . . . 12
  - 3.2.4 Discussion . . . 13
- 3.3 Performance vs. Computational Cost . . . 13

3.3.1 Code snippet . . . 13
3.3.2 Output . . . 14
3.3.3 Discussion . . . 14

4 Hyperparameters and Computational Cost 15
4.1 The Impact of Extended Training . . . 15
4.1.1 Code snippet . . . 15
4.1.2 Output . . . 15
4.1.3 Loss Plot . . . 15
4.1.4 Discussion . . . 15
4.2 Batch Size Dynamics . . . 16
4.2.1 Code snippet . . . 16
4.2.2 Output . . . 16
4.2.3 Loss Plot . . . 16
4.2.4 Discussion . . . 17
4.3 Advanced Optimization (Learning Rate & Schedulers) . . . 17
4.3.1 Code snippet . . . 17
4.3.2 Output . . . 18
4.3.3 Loss Plot . . . 19
4.3.4 Discussion . . . 19

NB: The contents within the table are hyperlinked for easier navigation of the document.

1 Task A: Binary Classification

1.1 Comparative Experiment

1.1.1 Updated plot snippet

```
# 4. Visualize - Altered plotting
plt.figure(figsize=[8,6], dpi=600) # Change dpi for clearer plot
plt.plot(loss_adam[0], "-", label="Adam - Train", color='blue')
plt.plot(loss_adam[1], label="Adam - Validation",color= 'green') # Add val loss

plt.plot(loss_sgd[0], "-", label="SGD - Train", color='orange')
plt.plot(loss_sgd[1], label="SGD - Validation", color ='red') # Add val loss

plt.legend(ncol=2, loc="upper center")
plt.title("Binary Classification Loss")
plt.xlabel("Epochs")
plt.ylabel("BCE Loss")
plt.ylim([1e-8, 10])
plt.yscale("log")
plt.grid()
plt.savefig("part1_loss.png", dpi=600) # Save figure
plt.show()
```

1.1.2 Training and Validation loss plot

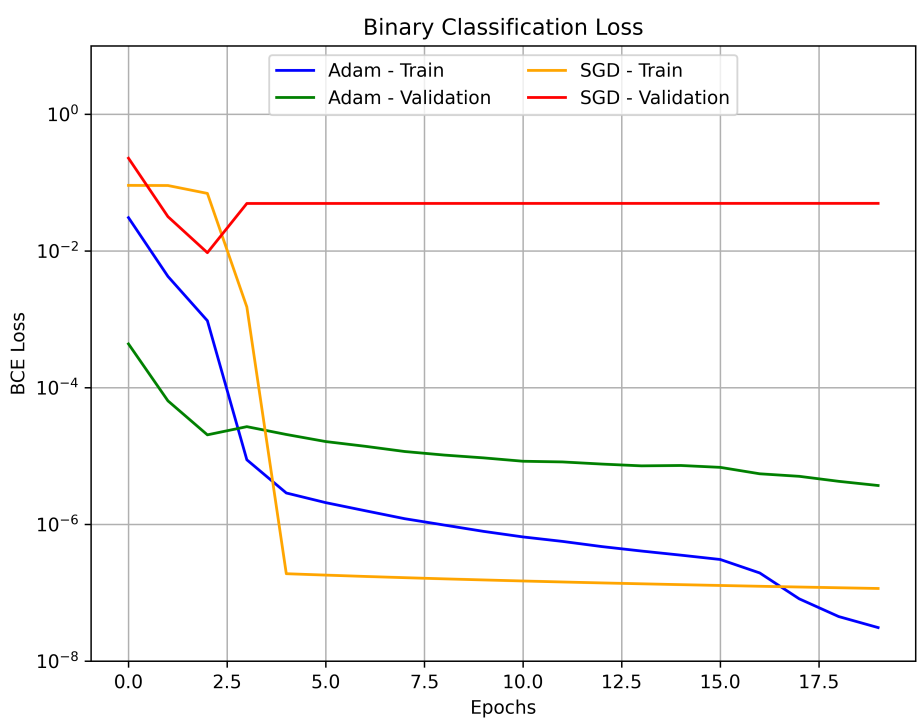


Figure 1: Training and Validation loss for both Adam and Stochastic Gradient Descent (SGD) optimised training.

1.1.3 Test loss snippet

```python
# CODE FOR TEST LOSS - ANSWER

def test_loss(model, loss_function, X_test, Y_test, batch_size):
    model.eval() # Use evalution mode from class
    total_loss = 0.0
    correct = 0
    total = 0

    with tch.no_grad(): # Don't calculate gradients
        for i in range(0, len(X_test), batch_size):
            X_batch = X_test[i:i + batch_size]
            Y_batch = Y_test[i:i + batch_size]

            outputs = model(X_batch)
            loss = loss_function(outputs, Y_batch)
            total_loss += loss.item()

    avg_loss = total_loss / ((len(X_test) + batch_size - 1) // batch_size)

    return avg_loss

test_loss_value_adam = test_loss(model_adam, loss_function, X_test, Y_test, batch_size)
test_loss_value_sgd = test_loss(model_sgd, loss_function, X_test, Y_test, batch_size)

print(f"Adam:")
print(f"Test loss: {test_loss_value_adam:.8f}")
print(f"\n")
print(f"SGD:")
print(f"Test loss: {test_loss_value_sgd:.8f}")

with open("p1test.txt", "w") as f:
    f.write(f"Adam:\n")
    f.write(f"Test loss: {test_loss_value_adam:.8f}\n")
    f.write(f"\n")
    f.write(f"SGD:\n")
    f.write(f"Test loss: {test_loss_value_sgd:.8f}\n")
```

1.1.4 Test loss output

```
Adam:
Test loss: 0.00000441

SGD:
Test loss: 0.02323385
```

1.2 Discussion

The Adam optimiser shows a lower test loss on the network than SGD, showing better performance with the high dimensionality of the data and the noise within the classes. Adam's consideration of momentum is likely the reason for better handling of the noise (due to better handling of large local gradients in the loss).

Figure 1 does suggest the network may be overfitting when trained with SGD given the validation loss is much larger than the training loss, and is not improving with further training, thus the network is not generalizing as well to new data compared with when training with Adam. This is supported by the test loss, considering the test loss is larger for SGD.

The SGD-trained network converges in fewer iterations than the Adam-trained network meaning there is a lesser compute cost, although as discussed there is a greater test loss suggesting the Adam-trained network provides better accuracy. The Adam-trained network still has decreasing loss after the 20 epochs, and isn't showing signs of underfitting or overfitting, thus will be more accurate but with much greater compute cost given an extra ≈ 12 epochs are requires to reach the same training loss.

2 Task B: Multiclass Classification

2.1 Increasing Class Density

2.1.1 Data preparation and training changes snippet

To include the number of classes (or centres) in the naming of the output file, num_classes is defined globally (equal to the number of centres which is also defined globally), additionally the number of classes is changed after initial run with 4 classes.

```
# 1. Prepare Data - Updated (centers=20, cluster_std=10)
# Note: y is NOT reshaped to (-1, 1) here because CrossEntropyLoss expects a 1D array of class
    indices
samples = 5000
centers = 20
X_mc, y_mc = make_blobs(n_samples=samples, centers=centers , n_features=64, cluster_std=10,
    random_state=2020)
... Other code here unchanged

# 3. Train
num_classes=20 # CHANGED: Defined globally for use in .txt file naming
model_mc = MulticlassNN(num_classes)
... Other code here unchanged
```

2.1.2 Evaluation snippet

The final training and validation is reported in the output and the output is saved to a .txt file for use in this document.

```
# 4. Evaluation - ADDED training and validation loss + save to .txt
# Compute the proportion of the test set for which we predict the correct class
# To get predictions, we take the argmax of the logits
loss_history_mc[-1].eval()
with tch.no_grad():
    logits = model_mc(X_test_mc)
    predictions = tch.argmax(logits, dim=1)
    accuracy = (predictions == Y_test_mc).float().mean()
    print(f"Test Accuracy: {accuracy:.4f}")
    #---------------- Added code for loss
    print(f"Final Training loss: {loss_history_mc[0][-1]:.6f}")
    print(f"Final Validation loss: {loss_history_mc[1][-1]:.6f}")
    with open(f"p2out{num_classes}.txt", "w") as f: # Save to .txt, use class no. for file name
        f.write(f"Test Accuracy: {accuracy:.4f}\n")
        f.write(f"Final Training loss: {loss_history_mc[0][-1]:.6f}\n")
        f.write(f"Final Validation loss: {loss_history_mc[1][-1]:.6f}")
```

2.1.3 Output

centers = 4

```
Test Accuracy: 0.9960
Final Training loss: 0.0000000513
Final Validation loss: 0.028410
```

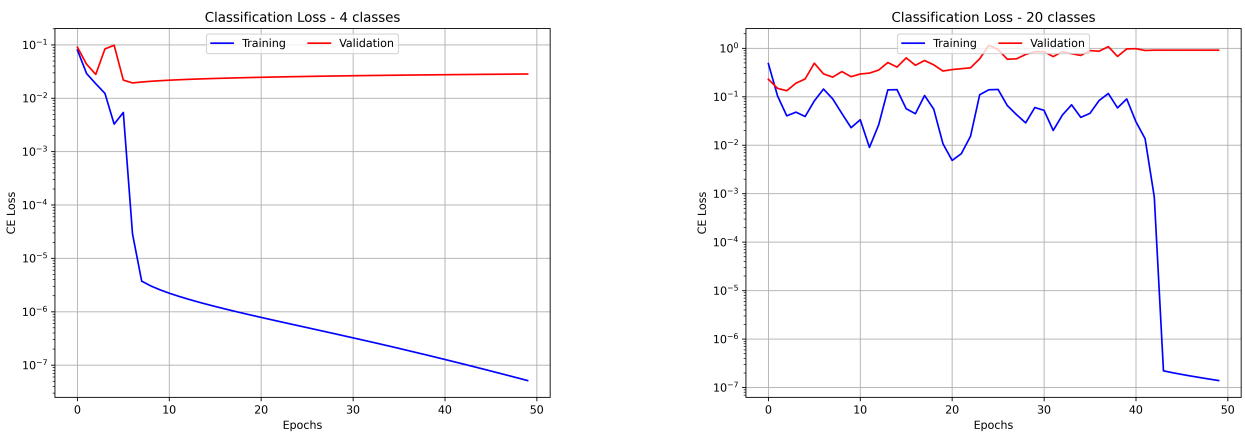
centers = 20

```
Test Accuracy: 0.9470
Final Training loss: 0.0000001391
Final Validation loss: 0.912983
```

2.1.4 Discussion

For all the outputs considered, the network performs significantly worse with 20 centres when compared with the same model with 4 centres. There is a 5% decrease in accuracy and an increase of approximately 1 order in both the training and validation losses. Additionally, the validation loss is increasing for both cases, suggesting overfitting. With 20 centres, this is significantly worse, with both the training and validation loss experiencing oscillatory convergence, as can be seen in figure 2. This is likely due to the difficulty of parameter tuning with the larger number of parameters.

Increasing the number of classes makes the classification task more difficult as there are more boundaries of the classes to consider which are likely to be more densely arranged as the class number is increased, making it more difficult for the network to differentiate classes and distinguish the margins between classes, causing a greater loss due to overlapping of some predictions.



(a) (b)

Figure 2: Training and Validation loss of the network when trained for 4 centres (a), and for 20 centres (b) - plotting code from Task A used.

2.2 Increasing Data Noise (Standard Deviation)

2.2.1 Updated data code snippet

The cluster_std is changed within the code to 20, all other code remains unchanged.

```
# 1. Prepare Data - Updated (centers=20, cluster_std=20)
# Note: y is NOT reshaped to (-1, 1) here because CrossEntropyLoss expects a 1D array of class
    indices
samples = 5000
centers = 20
X_mc, y_mc = make_blobs(n_samples=samples, centers=centers , n_features=64, cluster_std=20,
    random_state=2020)
... Other code unchanged
```

2.2.2 Output

```
Test Accuracy: 0.4630
Final Training loss: 0.0811498281
Final Validation loss: 13.544628
```

2.2.3 Loss Plot

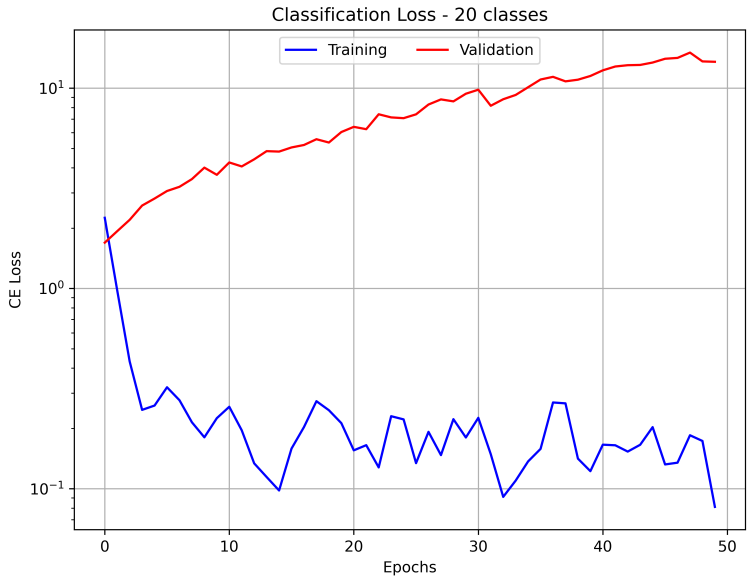


Figure 3: Training and Validation loss for noisier data (cluster_std=20).

2.2.4 Discussion

With the noisier data, the network performs much worse. Both the training and validation loss were much higher, with the validation loss increasing for the duration of training (as seen in figure 3), showing poor generalisation to the data and overall poor fitting to the data with the relatively high loss and low accuracy. The worsening is very clear when comparing the lower standard deviation data, seen in figure 2b. Increasing the noise in the data reduces the class separability and increases the overlap in classes, thus making it harder for the network to differentiate and classify the clusters.

2.3 High-Dimensional Input Space

2.3.1 Altered code snippet

The number of features (n_features) is defined globally to change the input argument to the first layer of the network. The rest of the network remains unchanged.

```
# 1. Prepare Data - Updated (centers=20, cluster_std=20, n_features=128)
# Note: y is NOT reshaped to (-1, 1) here because CrossEntropyLoss expects a 1D array of class
    indices
samples = 5000
centers = 20
n_features = 128 # Global definition for model change
```

```
X_mc, y_mc = make_blobs(n_samples=samples, centers=centers , n_features=n_features,
    cluster_std=20, random_state=2020)
... Other code here unchanged

# 2. Define Multiclass Model
class MulticlassNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(n_features, 256)
... Other code unchanged
```

2.3.2 Output

```
Test Accuracy: 0.8040
Final Training loss: 0.0000187022
Final Validation loss: 1.116927
```

2.3.3 Loss Plot

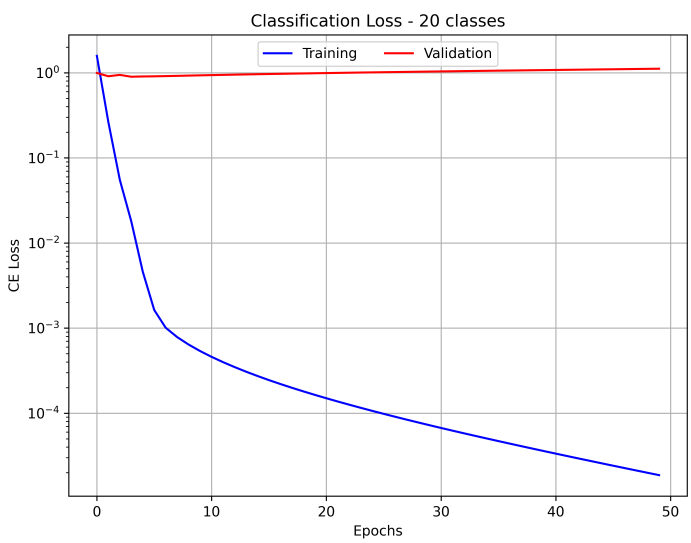


Figure 4: Training and Validation loss for higher dimensional input (n_features=128).

2.3.4 Discussion

Increasing the number of features showed significant improvement of the training of the network, although, the validation loss of the network was still increasing throughout the training of the network. This is again suggestive of overfitting although the training is much more successful than the original number of features. Increasing the number of features adds more dimensional information to the data, thus can improve the performance of the network as separation of the clusters become easier given more distinct boundaries in the clusters making it easier to more accurately separate the clusters. This can sometimes hinder the model if there is not enough distinct dimensional separability to the data, in which training may become harder given there is more information with only the same amount of useful information, this can also increase the risk of overfitting.

3 Task C: Model Architecture (Regression)

3.1 Activation Function Impact

3.1.1 Code snippet

Alteration for the GELU network:

```
# 2b. Define Regression Model - GELU
class GELURegressionNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(64, 256)
        self.gelu1 = nn.GELU()
        self.fc2 = nn.Linear(256, 1024)
        self.gelu2 = nn.GELU()
        self.out = nn.Linear(1024, 1)
        # No Final Activation (Linear)

    def forward(self, x):
        op = self.gelu1(self.fc1(x))
        op = self.gelu2(self.fc2(op))
        op = self.out(op)
        return op
```

Altered training code:

```
# 3. Train
model_relu = RegressionNN()
model_gelu = GELURegressionNN()
loss_fn_reg = nn.MSELoss()
optimizer_relu = tch.optim.Adam(model_relu.parameters(), lr=0.0001)
optimizer_gelu = tch.optim.Adam(model_gelu.parameters(), lr=0.0001) # Requires seperate
    optimizer definition
batch_size = 32
print("Training Regression Network (ReLU)...")
num_epochs = 150
loss_relu = train_network(model_relu, optimizer_relu, loss_fn_reg, num_epochs, batch_size,
    X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
print("Training Regression Network (GELU)...")
loss_gelu = train_network(model_gelu, optimizer_gelu, loss_fn_reg, num_epochs, batch_size,
    X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
```

Reporting and Visualisation (addition):

```
with open(f"p3out.txt", "w") as f: # Save to .txt
    f.write("ReLU:\n")
    f.write(f"Final Training loss: {loss_relu[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_relu[1][-1]:.4f}\n")
    f.write("GELU:\n")
    f.write(f"Final Training loss: {loss_gelu[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_gelu[1][-1]:.4f}")

# 4. Visualisation
plt.figure(figsize=(8, 5))

# ReLU model
```

```
plt.plot(loss_relu[0], label="ReLU - Train", linestyle="-", color="blue")
plt.plot(loss_relu[1], label="ReLU - Validation", linestyle="--", color="blue")

# GELU model
plt.plot(loss_gelu[0], label="GELU - Train", linestyle="-", color="orange")
plt.plot(loss_gelu[1], label="GELU - Validation", linestyle="--", color="orange")

plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Regression Training and Validation Loss")
plt.ylim(0,200)
plt.xlim(0,150)
plt.legend(ncol=2, loc="upper center")
plt.grid(True)
plt.savefig("p3loss.png", dpi=600)
plt.tight_layout()
plt.show()
```

3.1.2 Output

```
ReLU:
Final Training loss: 2.8710
Final Validation loss: 44.8926
GELU:
Final Training loss: 1.7415
Final Validation loss: 23.1116
```

3.1.3 Loss Plot

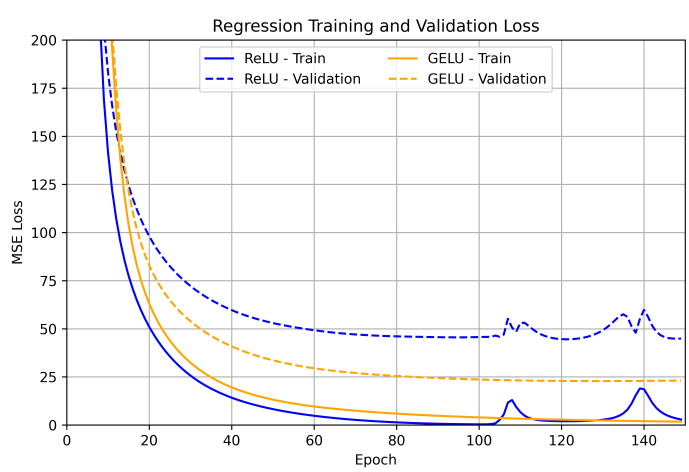


Figure 5: Training and Validation loss for ReLU and GELU regression networks.

3.1.4 Discussion

The ReLU and GELU networks both converge in similar time, with the GELU network showing better generalisation to the data (lower validation loss) as well as showing lower a lower training loss suggesting more accurate classification is achieved for the GELU network. The GELU network also experiences a

smoother loss curve without the occasional peaks in loss seen with the ReLU network. GELU provides more stable gradients and consequently better class/cluster separation thus the decision boundaries are easier to differentiate. Where ReLU ignores negative values, GELU does not thus for the ReLU network some parameters within the network can be excluded from training whereas GELU preserves these paramters so they are continuously trained even in the presence of negative values.

3.2 Increasing Model Depth

3.2.1 Code snippets

The new model DeeperRegression() is defined:

```
## TASK C
#Define Deep Regression Model
class DeeperRegression(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(64, 256)
        self.gelu1 = nn.GELU()
        self.fc2 = nn.Linear(256, 1024)
        self.gelu2 = nn.GELU()
        self.fc3 = nn.Linear(1024, 1024)
        self.gelu3 = nn.GELU()
        self.out = nn.Linear(1024, 1)
        # No Final Activation (Linear)

    def forward(self, x):
        op = self.gelu1(self.fc1(x))
        op = self.gelu2(self.fc2(op))
        op = self.gelu3(self.fc3(op))
        op = self.out(op)
        return op
```

Which is then trained:

```
# Train
model_deep = DeeperRegression()
loss_fn_reg = nn.MSELoss()
optimizer_deep = tch.optim.Adam(model_deep.parameters(), lr=0.0001)
batch_size = 32
print("Training Regression Network (Deeper)...")
num_epochs = 150
loss_deep = train_network(model_deep, optimizer_deep, loss_fn_reg, num_epochs, batch_size,
    X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
```

Then reported and plotted using the same code as seen previously in experiment 1.

3.2.2 Output

```
Deeper Regression:
Final Training loss: 0.4871
Final Validation loss: 33.1277
```

3.2.3 Loss Plot

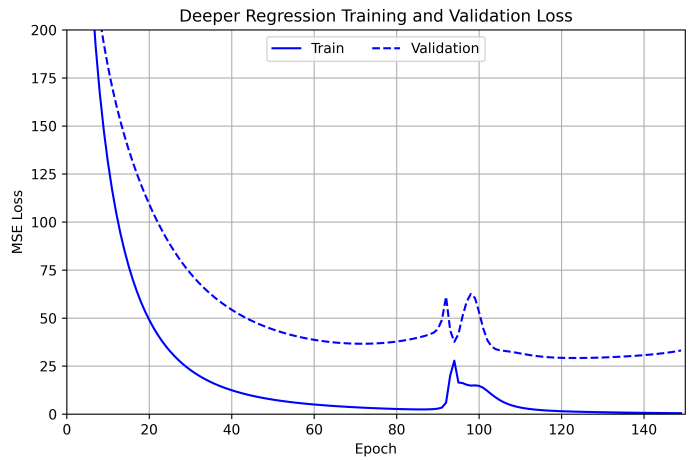


Figure 6: Training and Validation loss for the deeper network (DeeperNetwork() class).

3.2.4 Discussion

When compared with the GELU network, the DeeperRegression network experiences a lower training loss, however, has a higher validation loss. This suggests the network is better representing the seen data but not generalising as well as the GELU network. The increasing validation loss after ≈ 120 epochs suggests the DeeperRegression network is overfitting to the training data. Overall, the GELU network is better as it shows better generalisation given the smaller validation loss despite the DeeperRegression network having a lower training loss.

3.3 Performance vs. Computational Cost

3.3.1 Code snippet

The training was conducted again to find the overall time across the 150 epochs for a more accurate epoch time average:

```
# ReLU model
start = time.time()
loss_relu = train_network(model_relu, optimizer_relu, loss_fn_reg, num_epochs, batch_size,
    X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
avg_relu_time = (time.time() - start) / num_epochs

# Deeper model
start = time.time()
loss_deep = train_network(model_deep, optimizer_deep, loss_fn_reg, num_epochs, batch_size,
    X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
avg_deep_time = (time.time() - start) / num_epochs
```

And then reported with the following:

```
print(f"Avg epoch time (ReLU): {avg_relu_time:.4f} s")
print(f"Avg epoch time (Deep): {avg_deep_time:.4f} s")

with open("p3epoch_times.txt", "w") as f:
    f.write(f"ReLU: {avg_relu_time:.4f} s/epoch\n")
    f.write(f"Final Training loss: {loss_relu[0][-1]:.4f}\n")
```

```
    f.write(f"Final Validation loss: {loss_relu[1][-1]:.4f}\n")
    f.write(f"Deep: {avg_deep_time:.4f} s/epoch\n")
    f.write(f"Final Training loss: {loss_deep[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_deep[1][-1]:.4f}\n")
```

3.3.2 Output

```
ReLU:  0.1362 s/epoch
Final Training loss: 3.5186
Final Validation loss: 53.8579
Deep:  0.7172 s/epoch
Final Training loss: 0.4871
Final Validation loss: 33.1277
```

3.3.3 Discussion

The DeeperRegression network reduces the validation loss by $\approx 40\%$ at the cost of a $\approx 6-7\times$ increase in the computational cost. This loss is still relatively high and the network appears to be overfitting. This does not seem worth the added computational cost, especially given the GELU network can complete training in similar time to the ReLU network with a lower validation loss than the DeeperRegression network.

4 Hyperparameters and Computational Cost

4.1 The Impact of Extended Training

4.1.1 Code snippet

New instance of the model defined for training with further epochs to compare:

```
model_gelu_ext = GELURegressionNN() # New instance of the same network
loss_fn_reg = nn.MSELoss()
optimizer_gelu_ext = tch.optim.Adam(model_gelu_ext.parameters(), lr=0.0001)
batch_size = 32
print("Training Regression Network (GELU)...")
loss_gelu_ext = train_network(model_gelu_ext, optimizer_gelu_ext, loss_fn_reg, num_epochs,
    batch_size, X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
```

The same code from earlier was used for the output and plotting.

4.1.2 Output

```
GELU (250 epochs):
Final Training loss: 0.4931
Final Validation loss: 24.4385
```

4.1.3 Loss Plot

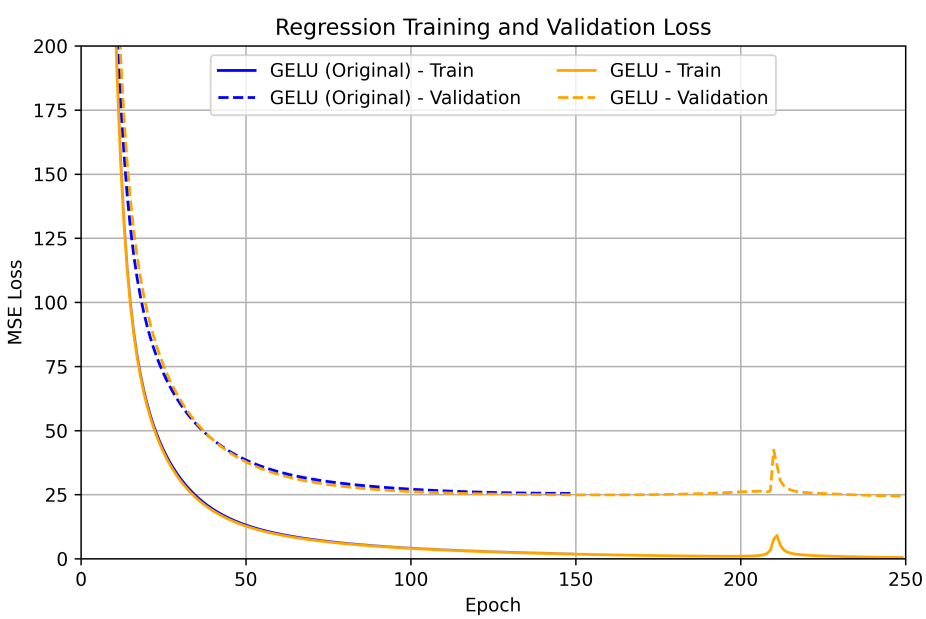


Figure 7: Training and Validation loss for the GELU network with additional iterations, with the original training losses superimposed.

4.1.4 Discussion

Extended training of the GELU network showed little improvement from the original training and validation losses (seen in 3.1.2) with the training loss roughly halved but the validation loss increasing slightly.

This suggests the further training is causing the network to more accurately represent the seen data but with no improvement in the generalisation of the network. From the loss curve (figure 7) the losses can be seen to plateau with very little change occuring after the 150 epochs.

4.2 Batch Size Dynamics

4.2.1 Code snippet

The batch size was changed to 256 and the original 150 epochs used, adding total time calculation used earlier:

```
model_gelu_ext = GELURegressionNN() # New instance of the same network
loss_fn_reg = nn.MSELoss()
optimizer_gelu_ext = tch.optim.Adam(model_gelu_ext.parameters(), lr=0.0001)
batch_size = 256
num_epochs = 150
print("Training Regression Network (GELU)...")
start = time.time()
loss_gelu_ext = train_network(model_gelu_ext, optimizer_gelu_ext, loss_fn_reg, num_epochs,
    batch_size, X_train_reg, Y_train_reg, X_val_reg, Y_val_reg)
avg_256batch_time = (time.time() - start) / num_epochs
```

The average times were added to the output, using the time and for the earlier 32 batch train for comparison:

```
with open(f"p4e2out.txt", "w") as f: # Save to .txt
    f.write("GELU (32 batch):\n")
    f.write(f"Average time per epoch: {avg_gelu_time:.4f} s/epoch\n")
    f.write(f"Final Training loss: {loss_gelu[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_gelu[1][-1]:.4f}\n")
    f.write("GELU (256 batch):\n")
    f.write(f"Average time per epoch: {avg_256batch_time:.4f} s/epoch\n")
    f.write(f"Final Training loss: {loss_gelu[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_gelu[1][-1]:.4f}\n")
... Same plotting code used
```

4.2.2 Output

```
GELU (32 batch):
Average time per epoch: 0.1808 s/epoch
Final Training loss: 1.9727
Final Validation loss: 23.4662
GELU (256 batch):
Average time per epoch: 0.0667 s/epoch
Final Training loss: 28.9100
Final Validation loss: 78.5871
```

4.2.3 Loss Plot

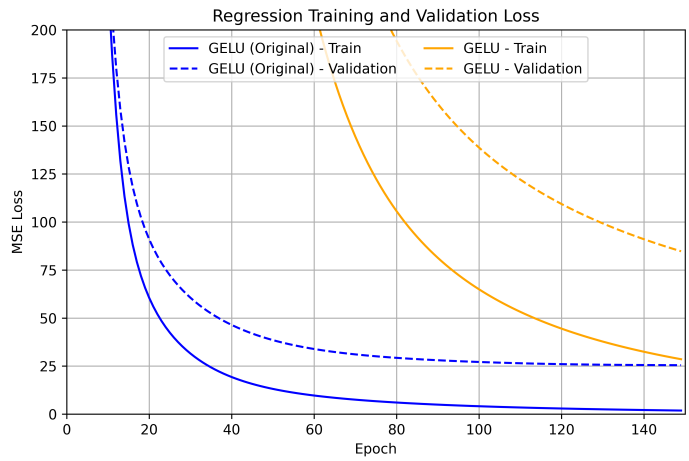


Figure 8: Training and Validation loss for the GELU network with 256 batch size, and the original 32 superimposed.

4.2.4 Discussion

Reducing the batch size causes the model to train much quicker ($\approx 3\times$ quicker) but has much larger loss for both training and validation. With further iteration it may converge to a smaller loss than it currently does after 150 epochs, although it is unlikely to decrease to a lower loss than the 32 batch training and further epochs would reduce the benefit of the reduced computational cost from using 256 batch size. The training loss is around $14\times$ larger and the validation loss around $3\times$ thus the reduction in validation loss is proportional to the computational cost reduction here although the training loss is much worse, thus the additional accuracy of the 32 batch is worth the incurred added cost.

4.3 Advanced Optimization (Learning Rate & Schedulers)

4.3.1 Code snippet

The train_network is changed to allow for scheduler to be used:

```
def train_network(model, optimizer, loss_function, num_epochs, batch_size, X_train, Y_train,
    X_val, Y_val, scheduler=None):
    """
    Generic training loop for PyTorch models.
    """
    model.train()
    loss_across_epochs = []
    loss_val_across_epochs = []

    for epoch in range(num_epochs):
        start_time = time.time() # <--- Start timer
        train_loss = 0.0



        ## TRAINING DATA
        # Manual batching loop
        for i in range(0, X_train.shape[0], batch_size):
            # Extract batch
```

```
            input_data = X_train[i:min(X_train.shape[0], i + batch_size)]
            labels = Y_train[i:min(X_train.shape[0], i + batch_size)]

            # Zero gradients
            optimizer.zero_grad()

            # Forward pass
            output_data = model(input_data)

            # Calculate loss
            loss = loss_function(output_data, labels)

            # Backpropagation
            loss.backward()

            # Update weights
            optimizer.step()

            train_loss += loss.item() * batch_size

        #Learning Rate Scheduler
        if scheduler is not None:
            scheduler.step()
... Rest of the training function unchanged
```

The following is then defined to train the DeeperRegression network with the learning rate scheduler:

```
model_deep_lrs = DeeperRegression()
loss_fn_reg = nn.MSELoss()
optimizer_deep_lrs = tch.optim.Adam(model_deep_lrs.parameters(), lr=0.0001)
scheduler = tch.optim.lr_scheduler.StepLR(optimizer_deep_lrs, step_size=25, gamma=0.5)
batch_size = 32
print("Training Regression Network (Deeper)...")
num_epochs = 150
loss_deep_lrs = train_network(model_deep_lrs, optimizer_deep_lrs, loss_fn_reg, num_epochs,
    batch_size, X_train_reg, Y_train_reg, X_val_reg, Y_val_reg, scheduler=scheduler)
```

The results of this train are then compared with that of the previous.

```
with open(f"p4e3out.txt", "w") as f: # Save to .txt
    f.write("Deep (No scheduler):\n")
    f.write(f"Final Training loss: {loss_deep[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_deep[1][-1]:.4f}\n")
    f.write("Deep (With scheduler):\n")
    f.write(f"Final Training loss: {loss_deep_lrs[0][-1]:.4f}\n")
    f.write(f"Final Validation loss: {loss_deep_lrs[1][-1]:.4f}\n")
... Same visualisation used
```

4.3.2 Output

```
Deep (No scheduler):
Final Training loss: 0.4871
Final Validation loss: 33.1277
Deep (With scheduler):
Final Training loss: 3.5601
```

```
Final Validation loss: 29.0910
```

4.3.3 Loss Plot

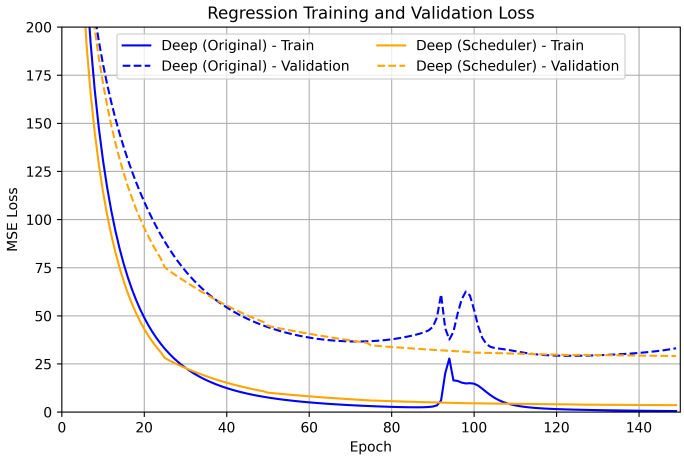


Figure 9: Training and Validation loss for the DeeperRegression network with and without learning rate scheduling.

4.3.4 Discussion

Addition of the learning rate scheduling causes smoothing of the loss curve in the later stages of training, and reduces the validation loss after the 150 epochs. The training loss, however, is higher but with the learning rate scheduler the network doesn't show signs of overfitting as with the previously trained network without the scheduler.